

Assignment 1: Producer-Consumer with a Bounded Buffer

Implement a producer-consumer scenario with a fixed-size (bounded) buffer. One or more producer processes generate data items and place them in the buffer. One or more consumer processes retrieve data items from the buffer. The producer(s) must wait if the buffer is full, and the consumer(s) must wait if the buffer is empty.

Coordinate the producers and consumers to avoid buffer overflow (producer writes to a full buffer) and buffer underflow (consumer reads from an empty buffer).

Hint:

Use shared memory for the buffer itself.

You'll need at least two synchronization primitives:

One to protect access to the buffer (making sure only one process modifies the buffer at a time). This could be a mutex or a semaphore.

One or more semaphores to signal the availability of empty slots in the buffer (for producers) and the availability of data items (for consumers). Consider two additional semaphores: `empty_count` and `full_count`.

Initialize these semaphores correctly. `empty_count` starts at buffer capacity, and `full_count` starts at 0.

Assignment 2: Readers-Writer Problem (Writer Priority)

Model a shared resource (e.g., a file) that can be accessed by multiple readers and writers. Multiple readers can access the resource concurrently, but only one writer can access it at a time, and no readers can access it while a writer is active. Implement writer priority: if a writer is waiting, it should be given preference over new readers. Ensure data consistency and correct access semantics while giving writers preference to avoid writer starvation.

Hint:

Use shared memory to store the shared resource (if applicable; you could also just simulate access).

You'll likely need multiple synchronization primitives:

A mutex or semaphore to protect access to shared data (e.g., a reader count).

A semaphore (`wrt`) that writers acquire exclusively.

Use a counter (`readcount`) to track the number of active readers. When the first reader enters, it acquires `wrt` so no writers can enter. When the last reader exits, it releases `wrt`.